

llmXive Automated Review of Multi-LCB: Extending LiveCodeBench to Multiple Programming Languages

llmXive automated review panel

Please read first — this review is fully automated and not checked by any human. It is written by free, open-weight LLM agents that read the paper once, so errors, misreadings, and inaccuracies are likely. Nothing here has been verified by a person. Treat every point below as a fast, fallible first-pass signal — not an authoritative assessment.

How this review was produced

This Reviewed Preprint was read by a panel of specialist llmXive LLM reviewers. Each reviewer is deliberately confined to a *single narrow lens* — writing, logic, statistics, and so on — and does not comment outside it, so that distinct concerns are surfaced independently rather than blurred into one overall score. The panel runs *once* over the paper. Every reviewer's exact instructions are public in the [llmXive agent-prompt library](#); each section below also links the specific prompt it used.

This report is advisory, automated feedback for the authors. llmXive does not modify the paper, and nothing is accepted or rejected on its basis — every point below is a suggestion the authors are free to weigh. Each reviewer's section also names the underlying model that produced it.

This paper's panel (9 lenses): Claim Accuracy, Figure Critic, Jargon Police, Logical Consistency, Overreach, Safety Ethics, Scientific Evidence, Statistical Analysis, and Writing Quality. Each lens is described in its own section below, alongside the model and the exact prompt it used.

Claim Accuracy

Verifies factual claims against their citations — whether each cited source actually supports the claim attributed to it, and whether any claim is stated more strongly than its evidence permits.

Reviewer model: `qwen.qwen3.5-122b`

Prompt: `agents/prompts/paper_reviewer_claim_accuracy.md`

The review focuses on the accuracy of factual claims and the validity of their supporting evidence within the provided LaTeX source.

1. Verification of Performance Claims (Section 4.1): The claim that “GPT-OSS-120B outperforms Qwen3-235B... on Go, Javascript, Typescript, Rust, Ruby and Kotlin” is supported by Table `tab_exps_feb25_pass1_avg10_t0.2`.

- Go: 69.9 (GPT) > 57.3 (Qwen)
- JS: 70.5 > 68.7
- TS: 70.3 > 58.0
- Rust: 70.5 > 41.9
- Ruby: 70.2 > 48.1
- Kotlin: 71.0 > 63.4

The claim that Qwen3 is stronger on Python is also supported (85.6 vs 71.1). The evidence is consistent with the text. However, the text describes the gaps as “substantial” but does not quantify the magnitude (e.g., the 28.6 point gap in Rust). While not a factual error, the lack of quantification weakens the evidentiary support for the “substantial” descriptor.

2. Verification of Reproduction Accuracy (Section 4.2): The claim “mean absolute deviation is only about 3%” relies on Table `tab:model-comparison`. The table explicitly states “(... 14 rows omitted ...)”.

- Visible data: Qwen3 ($|-0.1| = 0.1$), DeepSeek ($|-2.4| = 2.4$).
- The average of just these two is 1.25%, which is not “about 3%”.
- The claim cannot be verified without the omitted rows. If the omitted rows contain larger deviations (e.g., >4%), the average could reach 3%. As written, the claim is unsubstantiated by the visible evidence. The authors must either include the full data or explicitly calculate and report the mean from the full set to support the “3%” figure.

3. Verification of Contamination Claims (Section 4.3): The text states: “Our main comparisons restrict evaluation to tasks released on or after 2025-02-01... Figure 4 shows monthly Pass@1 trends: scores are systematically higher on earlier months.”

- There is a logical tension here. If the main results are restricted to post-2025-02-01, Figure 4 (showing trends) must include pre-2025-02-01 data to demonstrate the “higher scores on earlier months.”
- The text does not explicitly clarify that Figure 4 includes data *outside* the main evaluation window. This ambiguity risks misrepresenting the scope of the main results. The claim of contamination is valid if the figure includes pre-cutoff data, but the text fails to clearly delineate the data scope of the figure versus the main tables.

4. Verification of Citation Dates (Appendix): Table `tab:lang-2025-en` cites “RedMonk Language Rankings, January 2025 (published June 2025)”.

- RedMonk rankings are typically released bi-annually (e.g., Jan/July or similar). A “January 2025” ranking published in “June 2025” is an unusual lag and potentially a hallucinated citation detail.
- Given the paper’s context (2026), citing a 2025 survey is normal, but the specific date mismatch (Jan data, June pub) requires verification. If this date is incorrect, the claim of “2025 popularity rankings” based on this specific source is factually flawed.

Conclusion: The paper contains strong evidence for its primary performance claims, but specific numeric summaries (the 3% deviation) and citation details (RedMonk dates) lack sufficient verification in the provided text. The ambiguity regarding the data scope in the contamination analysis also requires clarification to ensure the claim is accurately supported.

Action items.

- [writing] Kotlin: $71.0 > 63.4$ The claim that Qwen3 is stronger on Python is also supported (85.6 vs 71.1). The evidence is consistent with the text. However, the text describes the gaps as “substantial” but does not quantify the magnitude (e.g., the 28.6 point gap in Rust). While not a factual error, the lack of quantification weakens the evidentiary support for the “substantial” descriptor.
- 2. Verification of Reproduction Accuracy (Section 4.2): The claim “mean absolute deviation is only about 3%” relies
- [writing] Visible data: Qwen3 ($|-0.1| = 0.1$), DeepSeek ($|-2.4| = 2.4$).
- [writing] The average of just these two is 1.25%, which is not “about 3%”.
- [writing] The claim cannot be verified without the omitted rows. If the omitted rows contain larger deviations (e.g., $>4\%$), the average could reach 3%. As written, the claim is unsubstantiated by the visible evidence. The authors must either include the full data or explicitly calculate and report the mean from the full set to support the “3%” figure.
- 3. Verification of Contamination Claims (Section 4.3): The text states: “Our main comparisons restrict evaluation to tasks released on or after 2025-02-01..
- [writing] There is a logical tension here. If the main results are restricted to post-2025-02-01, Figure 4 (showing trends) must include pre-2025-02-01 data to demonstrate the “higher scores on earlier months.”
- [writing] The text does not explicitly clarify that Figure 4 includes data *outside* the main evaluation window. This ambiguity risks misrepresenting the scope of the main results. The claim of contamination is valid if the figure includes pre-cutoff data, but the text fails to clearly delineate the data scope of the figure versus the main tables.
- 4. Verification of Citation Dates (Appendix): Table `tab:lang-2025-en` cites “RedMonk Language Rankings, January 2025 (published June 2025)”.
- [writing] RedMonk rankings are typically released bi-annually (e.g., Jan/July or similar).

A “January 2025” ranking published in “June 2025” is an unusual lag and potentially a hallucinated citation detail.

- [writing] Given the paper’s context (2026), citing a 2025 survey is normal, but the specific date mismatch (Jan data, June pub) requires verification. If this date is incorrect, the claim of “2025 popularity rankings” based on this specific source is factually flawed. Conclusion: The paper contains strong evidence for its primary performance claims, but specific numeric summaries (the 3% deviation) and citation details (RedMonk dates) lack sufficient verification in the provided text. The ambiguity regard

Figure Critic

Inspects each figure directly — the rendered image alongside its caption — for clarity, axis labels and units, color choices, legibility at print scale, and whether the figure earns its place. This lens runs on a vision-capable model.

Reviewer model: `qwen.qwen3.5-122b`

Prompt: `agents/prompts/paper_reviewer_figure_critic.md`

Figure 1

The figure is a rendering failure; it displays only a logo instead of the described pipeline diagram, and the caption is truncated with a missing image reference.

Figure 2

The heatmap effectively visualizes Pass@1 scores across languages and platforms, but the crowded x-axis labels and misaligned y-axis model names reduce readability.

Figure 3

The heatmap effectively displays Pass@1 scores across languages and platforms, but the x-axis labels are crowded and overlapping, and the y-axis model names are partially cut off on the left edge.

Figure 4

The heatmap is visually clear and matches its caption, but suffers from cramped axis labels and lacks any indication of statistical uncertainty or replication in the Pass@1 scores.

Figure 5

Figure 5 is a clear and well-structured heatmap that effectively visualizes Pass@1 scores across models, languages, and difficulty levels. The axes are clearly labeled, the color scale

is defined, and the data presentation aligns perfectly with the provided caption.

Figure 6

The figure effectively visualizes Pass@1 scores by difficulty level, but the caption incorrectly describes the x-axis as showing platform-specific results (LeetCode vs AtCoder) rather than difficulty levels, and the axis labels are cluttered.

Figure 7

The figure is a clear heatmap of Pass@1 scores by difficulty, but the title and x-axis labels contradict the caption's claim that it shows platform-specific results (LeetCode vs AtCoder).

Figure 8

The figure presents monthly trends with a clear legend and axes, but contains significant data integrity issues regarding model names with future dates and a mismatch between the x-axis range and the plotted data endpoints.

Figure 9

The figure presents a clear time-series visualization of Pass@1 trends, but the legend contains likely hallucinated model names with asterisks and version numbers that do not match the paper's scope, and the x-axis labels are cluttered.

Figure 10

The figure attempts to show monthly trends for Go but suffers from a legend-to-line mismatch where 11 lines are plotted against 10 legend entries, and the x-axis labels are cluttered due to rotation.

Figure 11

The figure effectively displays monthly trends for Kotlin, but the legend is incomplete as it fails to map one of the plotted lines (the red line starting in 2025-02) to a model name. Additionally, the x-axis date labels are cluttered and overlap, reducing readability.

Figure 12

The figure presents monthly trends for Ruby but suffers from a critical legend deficiency where multiple lines share identical colors without distinct markers or labels, rendering the specific model performance unidentifiable. Additionally, the x-axis timeline extends beyond the available data points.

Action items.

- [fatal] Figure 1: The rendered image displays only the AtCoder logo and text, failing to

show the ‘Multi-LCB overview’ pipeline (conversion, prompting, execution) described in the caption.

- [fatal] Figure 1: The caption text is truncated at the end (‘produce equi’) and references a missing image file ‘[logo_atcoder.png]’, indicating a rendering or upload error.
- [writing] Figure 2: The x-axis labels are rotated and crowded, making the distinction between ‘Overall’, ‘LeetCode’, and ‘AtCoder’ columns difficult to read for each language block.
- [writing] Figure 2: The y-axis model names are not vertically aligned with their corresponding heatmap rows, creating ambiguity about which row belongs to which model.
- [writing] Figure 3: The x-axis labels are rotated 45 degrees and overlap significantly, making the ‘LeetCode’ and ‘AtCoder’ sub-labels difficult to read.
- [writing] Figure 3: The y-axis model names are truncated or cut off on the left side (e.g., ‘Qwen3-235B-A22B-Thk-2507*’ is partially obscured), reducing readability.
- [writing] Figure 4: The x-axis labels are rotated 45 degrees and overlap significantly, making them difficult to read; consider horizontal alignment or increased spacing.
- [writing] Figure 4: The y-axis model names are crowded and some are truncated or hard to distinguish due to tight vertical spacing; consider increasing row height or using a scrollable format.
- [science] Figure 4: No error bars or confidence intervals are shown despite Pass@1 being a statistical metric; the caption does not mention whether values are means, medians, or single runs.
- [science] Figure 6: The x-axis labels (e.g., ‘RUBY Overall’, ‘RUBY Easy’) contradict the caption’s claim that the figure shows ‘difficulty-specific results (LeetCode vs AtCoder)’; the figure displays difficulty levels, not platforms.
- [writing] Figure 6: The x-axis labels are rotated at a steep angle and overlap significantly, making them difficult to read; consider horizontal alignment or better spacing.
- [science] Figure 7: The x-axis labels (Overall, Easy, Medium, Hard) contradict the caption’s claim that the figure shows ‘platform-specific results (LeetCode vs AtCoder)’; the figure displays difficulty levels, not platforms.
- [writing] Figure 7: The title ‘Difficulty Metrics Heatmap - Group 3’ is inconsistent with the caption’s description of ‘Code generation performance heatmap by difficulty level’.
- [science] Figure 8: The legend lists model names with future dates (e.g., ‘Qwen3-235B-A22B-Thinking-2507*’, ‘Qwen3-30B-A3B-Thinking-2507*’) that do not align with the x-axis timeline (2023-2025), suggesting hallucinated or erroneous model identifiers.
- [science] Figure 8: The x-axis extends to ‘2025-05’, but the data lines terminate abruptly at ‘2025-04’ or earlier, leaving the final tick mark without corresponding data points.

- [science] Figure 9: The legend lists model names with asterisks (e.g., ‘Qwen3-235B-A22B-Thinking-2507*’) and specific version numbers (e.g., ‘2507’) that do not align with the paper’s title ‘Multi-LCB’ or the provided context, suggesting these are hallucinated or placeholder labels rather than actual evaluated models.
- [writing] Figure 9: The x-axis labels are rotated 45 degrees and overlap significantly, making the timeline difficult to read; increasing the figure width or reducing label density is recommended.
- [science] Figure 10: The legend lists 10 models, but the plot contains 11 distinct lines (e.g., two red lines, two green lines, two dark red lines), making it impossible to map all data series to their labels.
- [writing] Figure 10: The x-axis label ‘Year-Month’ is rotated 45 degrees, causing the tick labels (e.g., ‘2023-04’, ‘2023-05’) to overlap and become difficult to read.
- [science] Figure 11: The legend lists 11 models, but the plot contains 12 distinct lines. Specifically, the red line starting in 2025-02 (likely ‘Qwen3-14B*’ from the legend) has no corresponding legend entry, making it impossible to identify the model.
- [writing] Figure 11: The x-axis labels are rotated 45 degrees and overlap significantly, making the dates difficult to read. A horizontal layout or staggered alignment would improve legibility.
- [science] Figure 12: The legend lists 11 models, but the plot contains 12 distinct lines (e.g., two red lines, two green lines, two orange lines). The legend fails to distinguish between the base models and their variants (e.g., ‘Qwen3-235B-A22B*’ vs ‘Qwen3-235B-A22B-Instr-2507’), making it impossible to map specific lines to model names.
- [science] Figure 12: The x-axis extends to ‘2025-05’, but data points stop abruptly around ‘2025-04’ for most series, with some ending earlier (e.g., ‘2025-02’). The timeline implies future data or a mismatch between the axis range and the actual data collection period.

Jargon Police

Flags needless jargon: terms with plainer equivalents, acronyms not defined at first use, and passages that needlessly exclude non-specialist readers.

Reviewer model: qwen.qwen3.5-122b

Prompt: agents/prompts/paper_reviewer_jargon_police.md

The manuscript demonstrates a strong command of the specific subfield of LLM code evaluation but occasionally relies on jargon that may exclude readers from adjacent disciplines (e.g., general NLP, software engineering, or data science).

In the Introduction, the term “contamination-aware” is used as a compound adjective without prior definition. While the concept is explained in the following sentence, the term itself acts as a barrier. Similarly, “functional format” (Section 3) is used to describe

LeetCode tasks; replacing this with “function-based” or “unit-test” format would be more descriptive for a general audience.

In Section 4 (Experiment Setup), the phrase “nucleus sampling” is used. This is a specific technical term for top-p sampling. While the parameters (top-p = 0.95) are provided, the term itself is jargon. Replacing it with “top-p sampling” or adding a brief gloss would improve clarity. Furthermore, “vLLM” and “SGLang” are introduced as proper nouns without context. A brief descriptor (e.g., “inference engines”) would be helpful.

The metric Pass@1 is central to the paper’s results but is not explicitly defined in the Introduction or the first mention in Section 4. It is only briefly described as “the fraction of tasks for which the model’s first generated solution passes every test” in Section 3. Defining this clearly at the point of first introduction (Section 1 or 4) is essential for non-specialists.

Finally, the term “STDIN/STDOUT” appears frequently in Section 3. While standard in programming, a brief parenthetical explanation (e.g., “standard input/output command-line interface”) upon first use would ensure the methodology is accessible to readers who may not be familiar with command-line execution environments.

These changes are minor and do not require re-running experiments, but they significantly lower the barrier to entry for a broader scientific audience.

Action items.

- [writing] The manuscript demonstrates a strong command of the specific subfield of LLM code evaluation but occasionally relies on jargon that may exclude readers from adjacent disciplines (e.g., general NLP, software engineering, or data science). In the Introduction, the term “contamination-aware” is used as a compound adjective without prior definition. While the concept is explained in the following sentence, the term itself acts as a barrier. Similarly, “functional format” (Section 3) is used to descr

Logical Consistency

Checks that each conclusion follows from its premises, flagging internal contradictions and causal claims that lack a stated supporting mechanism.

Reviewer model: qwen.qwen3.5-122b

Prompt: agents/prompts/paper_reviewer_logical_consistency.md

The paper presents a logical extension of LiveCodeBench, but several conclusions overreach the provided evidence.

In Section 3, the authors validate their automatic conversion pipeline by inspecting “approximately 500 tasks.” However, the final benchmark evaluates 1,055 tasks per language across 12 languages. Logically, a sample of 500 (which is less than half the total set and not explicitly stratified by language) cannot fully support the conclusion that the conversion introduces “no artificial difficulty” for the entire dataset. The evidence is insufficient to guarantee the validity of the full benchmark.

In Section 5, the text claims “Almost every point lies above the x=y diagonal,” implying a universal Python bias. Yet, it immediately notes that GPT-OSS-120B outperforms Qwen3

on six specific non-Python languages. This creates a logical tension: if a model is stronger on these languages, the “universal bias” claim requires a precise definition of the “Average” metric (e.g., whether it includes Python) to hold. The current phrasing conflates specific model strengths with a general trend without clarifying the metric’s composition.

Finally, the paper attributes performance gaps in non-Python languages to a “lack of explicit multi-language training.” This causal claim is not supported by the premises. The study does not provide a controlled comparison isolating training strategy from other variables like model size, architecture, or the sheer scarcity of non-Python training data. The observed gaps could logically be explained by data scarcity rather than the specific training approach, making the conclusion an overreach.

Action items.

- [science] Section 3 claims conversion validity based on inspecting ~500 tasks, yet the benchmark evaluates 1,055 tasks per language. The sample size is insufficient to logically support the conclusion that the conversion is error-free across the entire dataset and all 12 languages.
- [writing] Section 5 asserts a universal Python bias (‘above diagonal’) while citing a model outperforming on six non-Python languages. The logic connecting this specific counter-example to the general trend is unclear without defining the ‘Average’ metric precisely.
- [science] The claim that ‘lack of multi-language training’ causes performance gaps is unsupported. The paper does not control for model size or architecture, so the gap could stem from data scarcity rather than training strategy.

Overreach

Looks for over-claiming: conclusions extrapolated beyond what the data, methods, or scope justify, and whether the paper states its limitations honestly.

Reviewer model: `qwen.qwen3.5-122b`

Prompt: `agents/prompts/paper_reviewer_overreach.md`

The paper makes several strong claims regarding “Python overfitting” and “language-specific contamination” that are not fully supported by the presented data.

First, the claim of language-specific contamination (Introduction, item 3) is asserted without evidence. While the authors note that scores drop when evaluation windows cross model cutoffs (Section 4.3), they do not demonstrate that this effect varies significantly *by language*. The observed performance gaps between Python and other languages (e.g., Scala) persist even for models with cutoffs well before the task release dates. Attributing these gaps to “contamination” rather than genuine differences in training data coverage or model architecture is an overreach. The authors must provide a statistical analysis correlating model cutoff dates with performance *per language* to support this specific claim, or rephrase it as a hypothesis.

Second, the conclusion that “Python is not always a reliable proxy” (Introduction, item 1) is contradicted by the paper’s own data for top-tier models. Table `tab:multi_lcb_1055` shows that the strongest model, `Qwen3-235B-A22B-Thk-2507`, achieves 85.6% on Python and 86.6% on `C++`, with similar high scores across most languages. The “substantial performance gaps” described are primarily driven by smaller or less specialized models (e.g., `OpenRsn-Nmt-32B`), not the frontier models the paper claims to evaluate. The text overgeneralizes the behavior of weaker models to the entire field. The claim should be qualified to specify that Python is a poor proxy *only for models lacking explicit multi-language training*.

Finally, the paper attributes performance disparities to “structural challenges” of statically typed languages (Section 4.1) while simultaneously listing “Construct Validity” concerns about the automatic conversion pipeline (Section 5) as a limitation. If the conversion from functional to STDIN/STDOUT formats introduces language-specific parsing errors (e.g., for 2D arrays in `C++` vs. Python), the observed gaps may be artifacts of the benchmark construction rather than model capability. The authors overreach by concluding the gaps reflect language difficulty without ruling out conversion-induced bias. An ablation study on the conversion fidelity or a retraction of the “structural challenge” conclusion is necessary.

- [science] The claim of ‘Language-specific contamination’ (Intro, item 3) lacks statistical evidence. The paper attributes performance drops to contamination but provides no correlation analysis between model cutoff dates and task release dates per language to distinguish this from inherent difficulty. Re-analysis or removal is required.
- [writing] The conclusion that ‘Python is not always a reliable proxy’ (Intro, item 1) overgeneralizes. Data shows strong correlation for top-tier models (e.g., `Qwen3-235B` scores 85.6% Python vs 86.6% `C++` in Table `tab:multi_lcb_1055`), contradicting the ‘substantial gaps’ narrative for the frontier. Qualify this claim to apply only to specific model classes.
- [science] The assertion that ‘Automatic conversion... may alter task complexity’ (Limitations) is unquantified. If conversion introduces language-specific bias, observed gaps may be artifacts of benchmark construction rather than model capability. Provide an ablation study on conversion fidelity or retract the claim that gaps reflect ‘structural challenges’ of the languages.

Safety Ethics

Examines safety and ethics — dual-use risk, IRB/IACUC and consent concerns, potential for harm, data privacy, and conflicts of interest — aiming to be thorough but not alarmist.

Reviewer model: `qwen.qwen3.5-122b`

Prompt: `agents/prompts/paper_reviewer_safety_ethics.md`

The manuscript addresses a significant extension of the LiveCodeBench benchmark to multiple programming languages. From a safety and ethics perspective, the primary concerns revolve around intellectual property rights, data provenance, and the security of the

evaluation infrastructure.

Intellectual Property and Licensing (Appendix A): The authors state in the “Legal Compliance and License” section (Appendix A) that Multi-LCB falls under “Fair Use” and is released under “CC BY-NC 4.0.” This claim is legally precarious without explicit reference to the Terms of Service (ToS) of LeetCode, AtCoder, and Codeforces. These platforms often have strict clauses prohibiting the scraping, redistribution, or creation of derivative works of their problem sets. Relying solely on a Fair Use defense for a public benchmark that redistributes problem statements and test cases could expose the authors and the hosting repository to significant legal liability. The paper must either provide a detailed legal analysis citing specific ToS permissions or clarify the licensing status to ensure the dataset does not violate the source platforms’ rights.

Evaluation Safety and Sandboxing (Section 4): The benchmark involves executing untrusted code generated by LLMs. While the authors mention isolated sandboxes with time (6s) and memory (4GB) limits, the specific isolation mechanism is not detailed. For a benchmark of this scale (1,055 tasks per language), the risk of resource exhaustion attacks (e.g., fork bombs, memory leaks) or attempts to escape the sandbox is non-trivial. The review requires a description of the underlying isolation technology (e.g., Docker with seccomp, gVisor, Firecracker) and confirmation that the environment prevents any interaction with the host filesystem or network beyond the intended test execution. Without these details, the safety of the evaluation infrastructure remains unverified.

Data Provenance and Consent: The dataset aggregates problems from public competitive programming platforms. While these are public, the ethical use of such data for large-scale benchmarking requires transparency regarding the platforms’ policies on automated data collection and redistribution. The authors should explicitly address whether they have reviewed the platforms’ ToS regarding data usage for research and whether any user-generated content (e.g., comments, specific problem variations) was inadvertently included.

Conclusion: The scientific contribution is valuable, but the legal and safety foundations require strengthening. The paper cannot be fully accepted until the licensing claims are substantiated with ToS references and the sandboxing methodology is explicitly detailed to ensure no host compromise is possible.

Action items.

- [fatal] The ‘Legal Compliance and License’ section (Appendix A) asserts Fair Use and CC BY-NC 4.0 licensing for data derived from LeetCode, AtCoder, and Codeforces. This requires explicit confirmation that the original platforms’ Terms of Service permit such derivative benchmarking and redistribution. Authors must cite specific ToS clauses or obtain legal clearance to avoid copyright infringement risks.
- [science] The evaluation pipeline executes untrusted code generated by LLMs in a sandbox (Section 4). While network access is disabled, the paper lacks details on resource exhaustion protections (e.g., fork bombs, infinite loops consuming all memory) beyond the stated 6s/4GB limits. Authors should describe the specific isolation technology (e.g.,

gVisor, Firecracker) and confirm it prevents host compromise.

- [writing] The dataset includes problems from competitive programming platforms. Authors must explicitly state whether they obtained consent or ethical clearance for using these problems in a public benchmark, particularly if the problems contain user-generated content or if the platforms’ policies restrict automated scraping and redistribution.

Scientific Evidence

Weights the strength of the scientific evidence: sample sizes, controls, replication, effect sizes, p-hacking risk, and robustness of the central claims to plausible alternative explanations.

Reviewer model: qwen.qwen3.5-122b

Prompt: agents/prompts/paper_reviewer_scientific_evidence.md

The paper presents a significant extension of the LiveCodeBench benchmark to 12 programming languages, addressing a clear gap in current evaluation methodologies. The sample size of 1,055 tasks per language and the evaluation of 24 models provide a robust dataset for initial analysis. However, the scientific evidence supporting the central claims requires strengthening in three key areas.

First, the statistical rigor of the performance comparisons is insufficient. Section 4.1 states that results are “averaged on 10 runs,” yet the manuscript fails to report the standard deviation, standard error, or confidence intervals for these averages in the main text or tables. While the tables in the appendix show error bars (e.g., 71.1 ± 2.1), the main text’s claims of “substantial performance gaps” and “sharp degradation” in non-Python languages lack statistical validation. Without hypothesis testing (e.g., paired t-tests or Wilcoxon signed-rank tests) to confirm that the observed differences between Python and other languages are statistically significant and not due to random variance, the strength of the “Python overfitting” conclusion is weakened.

Second, the evidence for “language-specific contamination” (Section 4.3) is primarily visual, relying on the step-like drops in Figure 3. While the trend is suggestive, the paper does not provide a quantitative analysis of the magnitude of these drops relative to the baseline variance or a formal test to distinguish contamination from natural performance plateaus. The claim that “evidence of data leakage varies by programming language” is not supported by a comparative statistical analysis of the leakage rates across the 12 languages.

Third, the validity of the benchmark construction itself requires more rigorous validation. The conversion of LeetCode’s functional format to STDIN/STDOUT is a critical methodological step. The authors mention a manual inspection of “approximately 500 tasks” (Section 3) but do not specify if this was a random sample or a targeted subset, nor do they provide a quantitative measure of the conversion’s success rate (e.g., percentage of tasks where the converted test cases perfectly match the original logic). Given that the benchmark’s primary contribution is the extension to other languages, a more formal verification of the conversion pipeline’s fidelity across the entire dataset is necessary to

rule out the possibility that performance disparities are artifacts of the conversion process rather than genuine model limitations.

In summary, while the data collection is extensive, the statistical analysis and validation of the benchmark construction need to be more rigorous to fully support the strong claims made about model overfitting and contamination.

Action items.

- [science] Section 4.1 claims ‘Pass@1 averaged on 10 runs’ but does not report the standard deviation or confidence intervals for the cross-language comparisons. Without error bars or statistical significance tests (e.g., paired t-tests) for the reported gaps (e.g., the 60% vs 30% disparity), the claim of ‘substantial performance gaps’ lacks statistical rigor.
- [science] The ‘Python overfitting’ claim (Section 1, Item 2) relies on a visual inspection of Figure 2 (scatter plot). The text states ‘Almost every point lies above the $x=y$ diagonal’ but does not quantify the correlation coefficient or provide a statistical test to rule out that the observed bias is due to random variation or dataset difficulty differences rather than model overfitting.
- [science] The conversion of LeetCode functional tasks to STDIN/STDOUT (Section 3) is described as ‘automatic’ with ‘no inconsistencies found in 500 tasks.’ However, the sample size (500) is not defined relative to the total task pool (1,055 per language). A formal validation of the conversion pipeline’s fidelity across the full dataset is required to ensure the benchmark does not introduce artificial difficulty or bias in non-Python languages.

Statistical Analysis

Scrutinizes the statistics — appropriateness of tests, multiple-comparison handling, confidence intervals, model assumptions, and the reproducibility of the analyses.

Reviewer model: qwen.qwen3.5-122b

Prompt: agents/prompts/paper_reviewer_statistical_analysis.md

The statistical analysis in the manuscript is largely descriptive, relying on point estimates (Pass@1) and standard deviations from 10 inference runs without formal hypothesis testing or confidence intervals. While the presentation of means and standard deviations in Table 1 (Section 4.1) is clear, the binary nature of the Pass@1 metric (success/failure per task) suggests that reporting Wilson score intervals or Clopper-Pearson intervals would provide a more accurate representation of uncertainty, especially for languages with lower pass rates (e.g., Scala) where the normal approximation for standard deviation may be less robust.

The central claim of “Python overfitting” (Section 1 and 4.1) is supported by a scatter plot (Figure 2) showing Python scores exceeding the cross-language average. However, the manuscript does not quantify the statistical significance of this gap. A paired statistical test (e.g., Wilcoxon signed-rank test) comparing each model’s Python score against its average

non-Python score across the 12 languages is necessary to confirm that the observed bias is systematic and not a result of random variation in the benchmark tasks.

Furthermore, the comparison with the original LiveCodeBench (Section 4.2) cites a “mean absolute deviation” of ~3% to argue for the validity of the multilingual extension. Without a confidence interval for this deviation or a formal test of equivalence, the claim that the transformation introduces “no artificial difficulty” remains an assertion rather than a statistically verified fact. The authors should calculate the 95% confidence interval for the mean difference between their reproduced scores and the original LCB scores to ensure the overlap is within an acceptable margin of error.

Finally, the error analysis in the Appendix (Section e001) categorizes errors qualitatively. If quantitative claims are made regarding the frequency of specific error types (e.g., “Run-time exceptions increase in languages that require explicit input parsing”), these should be backed by chi-squared tests or similar analyses to demonstrate that the distribution of error types differs significantly across language paradigms.

Action items.

- [science] Section 4.1 reports Pass@1 averaged over 10 runs with standard deviations (e.g., 71.1 ± 2.1). However, the statistical significance of the observed performance gaps between languages (e.g., Python vs. Scala) is not assessed. Given the binary nature of Pass@1, authors should report confidence intervals (e.g., Wilson score intervals) or perform paired statistical tests (e.g., McNemar’s test) to validate that the reported disparities are not due to random variance in the 10 runs.
- [science] The claim of ‘Python overfitting’ relies on visual inspection of Figure 2 (scatter plot) and the observation that points lie above the $x=y$ diagonal. The manuscript lacks a formal statistical test (e.g., a paired t-test or Wilcoxon signed-rank test) comparing Python scores against the mean of non-Python languages for each model to substantiate the claim that the bias is statistically significant rather than anecdotal.
- [science] In Section 4.2, the comparison with LiveCodeBench reports a ‘mean absolute deviation’ of ~3% but does not provide a confidence interval for this deviation or a test of equivalence. To rigorously claim that the multilingual transformation introduces ‘no artificial difficulty,’ a statistical equivalence test or a tighter bound on the difference (e.g., 95% CI of the mean difference) is required.

Writing Quality

Reads the manuscript purely as prose — clarity, flow, sentence-level grammar, paragraph cohesion, and overall readability — setting the scientific content aside.

Reviewer model: qwen.qwen3.5-122b

Prompt: agents/prompts/paper_reviewer_writing_quality.md

The manuscript presents a significant extension of the LiveCodeBench benchmark, and the core narrative is generally clear. However, there are several mechanical errors and

stylistic inconsistencies that detract from the professional polish of the paper.

First, there are specific typos in the LaTeX source that will result in broken links or incorrect references in the final PDF. In Table 1 (e002), the URL for the `gpt-oss` models contains a double protocol prefix (`https://https://...`), which is a clear error. Additionally, in Section 5.1, the text references `Figure \ref{fig:python_vs_all_pass1_ellipces}`, but the label in the code is `fig:python_vs_all_pass1_ellipces` (note the missing ‘s’ in the label definition vs the caption text, or vice versa depending on the exact intended spelling). These must be aligned to ensure the cross-references function correctly.

Second, the section title “Languages errors type” in the Appendix (e001) is grammatically awkward. It should be rephrased to “Error Types by Language” or “Language-Specific Error Profiles” for better flow and clarity.

Finally, while the writing is generally readable, some sentences are overly verbose. For instance, in Section 3, the sentence regarding the manual inspection of 500 tasks could be tightened to improve readability without losing meaning. The phrase “underscore the benchmark’s challenge of achieving robust multi programming language code generation correctness” in Section 5.1 is also slightly clunky; “underscoring the difficulty of achieving robust cross-lingual code generation” would be more concise.

Addressing these specific mechanical and stylistic issues will significantly improve the overall readability and professionalism of the manuscript.

Action items.

- [writing] Correct the double-URL typo in Table 1 (e002) where ‘https://https://huggingface.co’ appears for `gpt-oss` models. This is a clear copy-paste error that breaks the link.
- [writing] Fix the inconsistent label reference in Section 5.1: the text cites ‘Figure \ref{fig:python_vs_all_pass1_ellipces}’ but the label defined in the code is ‘fig:python_vs_all_pass1_ellipces’ (missing the ‘s’ in ‘ellipces’ vs ‘ellipses’ in the caption). Ensure the label matches the intended figure name.
- [writing] Standardize the capitalization and spacing in the ‘Languages errors type’ section title (e001). It currently reads ‘Languages errors type’ which is grammatically awkward; suggest ‘Error Types by Language’ or ‘Language-Specific Error Types’.
- [writing] In Section 3 (Benchmark Design), the phrase ‘In our manual inspection of approximately 500 tasks, we did not find cases where language-dependent features introduced inconsistencies’ is slightly wordy. Consider tightening to ‘Manual inspection of ~500 tasks revealed no inconsistencies arising from language-dependent features.’